

— Command Line and Git

Learning Objectives

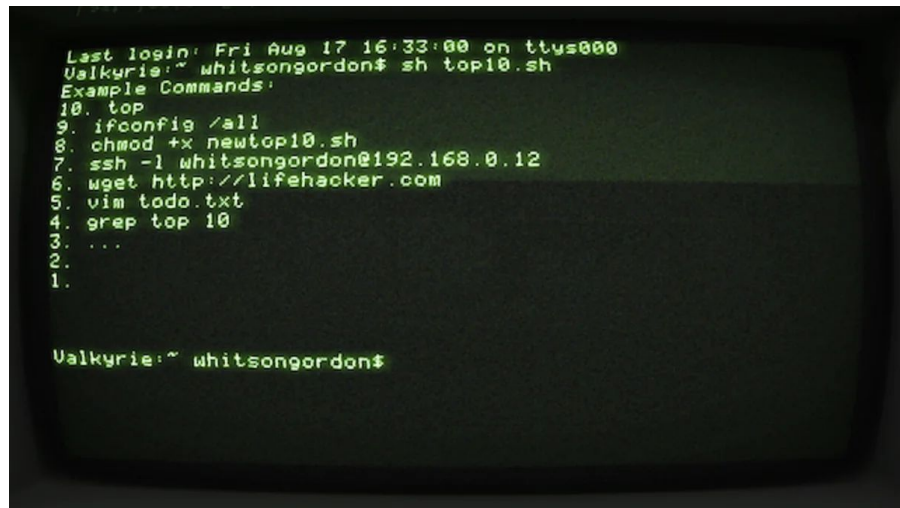
- Part 1: The UNIX Command Line
 - What is it and why use it?
 - Movin' around
 - Lookin' at things
 - Makin' things
- Part 2: Git
 - What is version control, and why use it?
 - The Git workflow
 - How **we** use Git in our class



CLI vs GUI

It used to be computers didn't have a **graphical user interface (GUI)**. You had to do everything via the **command line interface (CLI)**.

Today we take GUIs for granted. Most computer users, if they even know the command line exists, are confused and even a little scared by it. But not you!



```
Last login: Fri Aug 17 16:33:00 on ttys000
Valkyrie:~ whitsongordon$ sh top10.sh
Example Commands:
10. top
9. ifconfig /all
8. chmod +x newtop10.sh
7. ssh -l whitsongordon@192.168.0.12
6. wget http://lifelacker.com
5. vim todo.txt
4. grep top 10
3. ...
2.
1.

Valkyrie:~ whitsongordon$
```

https://i.kinja-img.com/gawker-media/image/upload/c_fill,f_auto,fl_progressive,g_center,h_675,pg_1,q_80,w_1200/17waftgjfrx4pjpg.jpg

What is the Command Line?

We'll use the CL for all sorts of things.



Why do you think it might be more beneficial to use a text-based CLI rather than a GUI?



<https://securecdn.pymnts.com/wp-content/uploads/2019/12/hacker-Apple-Turkish-NCA-investigation.jpg>

Terminal-ogy

- The **shell** is the specific command line language you're typing in
 - The go-to industry standard is **bash**, so that's what we'll use
 - ... unless you're using the newest version of MacOS, in which case you'll be using **Zshell**, which is nearly identical
 - For now, use whatever your computer has installed, and let us know if you experience any unexpected behavior
- The **terminal** is the program you use to emulate the shell
 - If you're on MacOS, that program is simply called **Terminal**, but another popular third party choice you can download is **iTerm2**
 - If you're on Windows, that program is **Git Bash**, although some experienced users might prefer the **Windows Subsystem for Linux (WSL)**

Let's get started!

Open up those terminals!



So you're dropped into a terminal...

Where are we?

pwd = print working directory (where am I right now?)

(We'll all see something different)



File Paths: root

In UNIX-based systems (ie, Mac and Linux), your folder's file tree begins at **root**. The **root directory** is the folder on your computer that contains *everything*. It's denoted simply by a **slash (/)**.

In Windows, Git Bash will emulate this, but it's not *really* true. Your C:\ drive is mounted in **/c**.

File Paths: home

Each user on a computer has a **home directory**. It is often denoted simply with a **tilde (~)**. When I typed, `pwd`, I saw...

c/Users/carys

home dir

root

Let's Walk and Talk

How do we move around our computer?

cd = change directory = move to a certain folder



Absolute Paths

You can **cd** into a specific directory if you know its *exact* location. The exact location, starting with root, is a folder's **absolute path**. For example:

```
cd /Users/dsi/Documents/project
```

or...

```
cd ~/Documents/project
```

(I'll be using 'dsi' as a generic user indicator in remaining slides)

Relative Paths

Sometimes you don't know the *exact* location of a folder, but you do know it *relative* to where you currently are. If this is the case, you can also use a **relative path**. For example, if we're currently in our home directory,

instead of

```
cd ~/Desktop/project
```

we can write

```
cd Desktop/project
```



Relative Paths

And again, if we're on our Desktop, we can write

```
cd project
```

But how do I get back up to my desktop? Or my home directory? The notation to “go up” a folder is the **double dot (..)**. And so, from inside this project folder, to go back up to the desktop, you would type:

```
cd ..
```

(go up one level)

And to go up to your home directory:

```
cd ../..
```

(go up two levels)

Shortcuts

cd (with no path given)	Go to home
cd ~	Go to home
cd ..	Go up
cd .	Do nothing (. = “this folder”)
cd -	Go back to last directory

Look Around

The **ls** command is used to **list** out all the files and folders in a given directory.



You Try:

- Look at all the items in your home directory
- Look at all the items on your Desktop
- Look at all the items in your root directory

— Makin' Stuff

Creating Directories

You can use the **mkdir** command to create a new directory.

Note: avoid putting spaces in file and folder names!



You Try:

Make a folder on your desktop called **tutorial** and then navigate into it.

Let's Explore our Development Environment

Make sure you're in this new directory. Let's run the **jupyter lab** command to execute the Jupyter Lab software.

Let's play around here and make a quick notebook.



Let's make files the “regular” way

You can use the **touch** command to make a blank file.



What kind of file do you think **touch** makes?

Let's make files the “regular” way

You can use the **touch** command to make a blank file.



Use **touch** to make a **hello.py** and **goodbye.py** file in our tutorial folder.

Let's Edit!

To edit a plain old text file, we'll need a **text editor**. There are millions of them out there, and experienced programmers debate which is best very fiercely. Right now, the three most popular ones are **Atom**, **VS Code**, and **Sublime**.

Depending on your text editor you might be able to open a text file directly using a command like **code <filename>** or **atom <filename>**.

Alternatively you can run **open .** to open the current location in Finder, your Mac's file browser GUI.



Hello, World!

Not only is Python a programming language... it's also a program! Actually, it's a program that runs programs! We can run this file with the **python** command (**python3** on some Linux and MacOS versions).

To run our program, we can simply run

```
python hello.py
```

But what if our file changes? How can we keep track...?



Man Pages

If you forget what a command does, you can always reference the **man page** for that command. (“Man” is short for “manual.”) Let’s open the man page for **ls**:

```
man ls
```

To exit the man page, tap **q** (for “quit.”)

Cheat Sheet

cd /path/to/folder	Change directory (move)
pwd	Print working directory (where am I now?)
ls	List everything where I am
mkdir folder-name	Make a folder called folder-name
touch file-name.py	Make an empty text file called file-name.py

Cheat Sheet

man <command>	View man page for <command>
ctrl + C	Abort current execution/line
 and 	View previously run commands
clear	Clear the terminal
python file-name.py	Run file-name.py using Python

— Version Control with Git

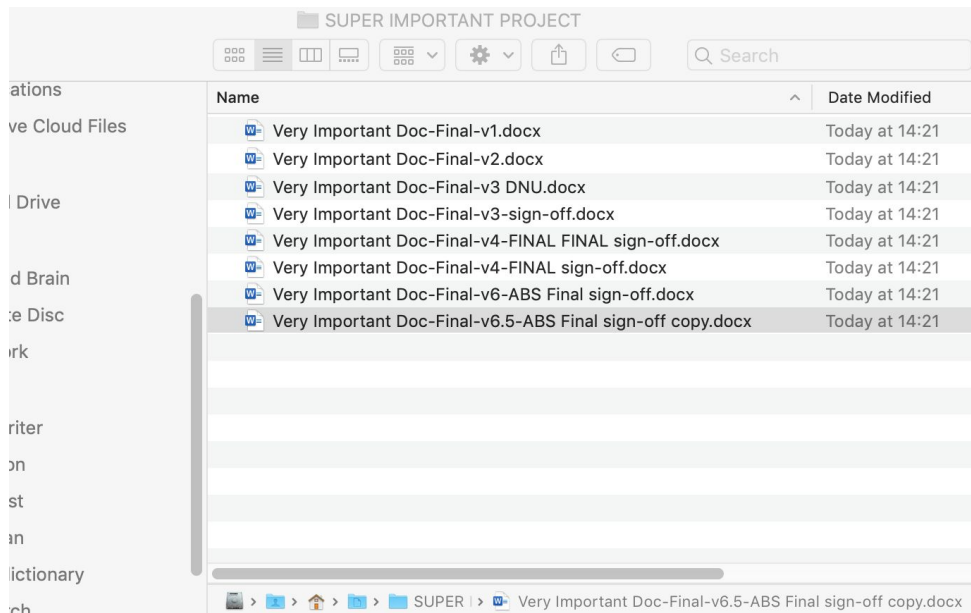
What is Git?

Git is a **version control software**. Why would you need such a thing?



What is Git?

Git is a **version control software**. Why would you need such a thing?



Screenshot from Charlie Rice

Git solves this (for text files)!

Git was created by legendary programmer **Linus Torvalds**, the same man who brought us Linux.

It was actually written in 2005 as a tool to help maintain the Linux kernel itself! He named it Git, since everyone was calling him a git (slang for a grumpy old man).

Linus + UNIX = Linux

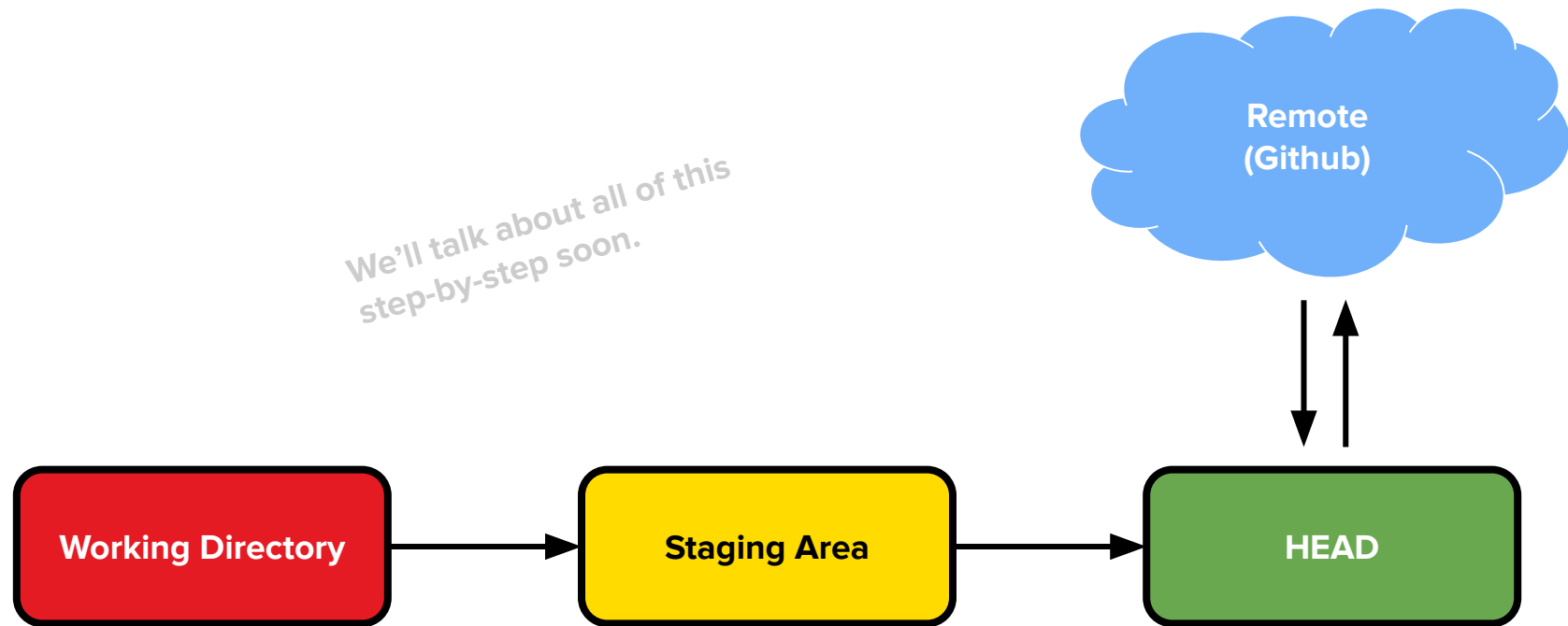


<https://cdn.britannica.com/99/124299-050-4B4D509F/Linus-Torvalds-2012.jpg>

Too Many Gits

Git	A distributed version control software
Git Bash	A very bad name for a Bash terminal emulator. A better name might be “Bash with Git”
Github	A popular website for hosting Git repositories. Think “Instagram for programmers”
Github Enterprise	A commercial version of Github. General Assembly pays a lot of money annually for our own private Github Enterprise server

The Git Workflow



First: Let's Wrangle a Repo

There are two ways to create a repository:

1. Start one from scratch via the command line (tedious, not for beginners)
1. Create one using Github and “clone” it onto our own machines



First: Let's Wrangle a Repo

There are two ways to create a repository:

1. Start one from scratch via the command line (tedious, not for beginners)

1. **Create one using Github and “clone” it onto our own machines**



Let's do that!

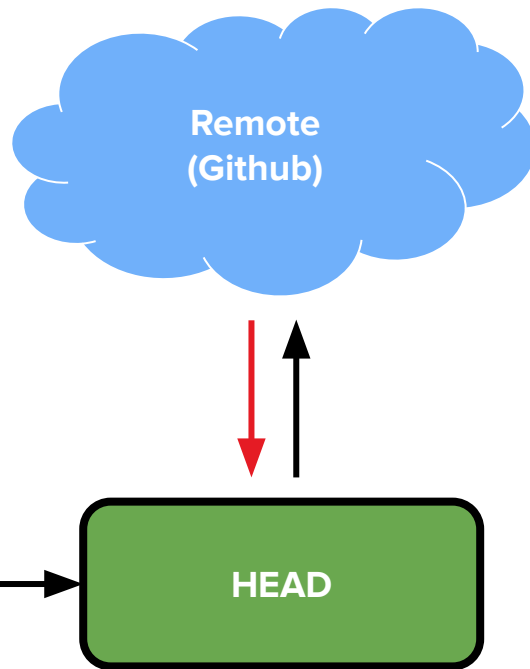
Let's create a repo named **my-first-repo** now!



Bringing Things Down to Local

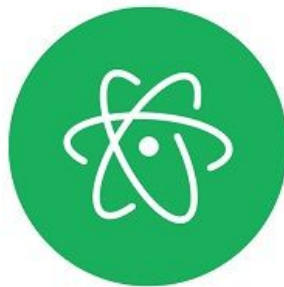
To get our repos locally, we'll have to download, or “clone” them from the remote:

git clone ...



Be the change you wish to see in the repo

Let's create a quick **greet.py** file in our repo.



Status Checking

At any point in this whole process, if we're in a repo, we can check what Git knows about by typing

git status



Staging Changes

We've just made a change to our **working directory**. The next step is to save this change by **staging it**.

git add .



Staging Changes

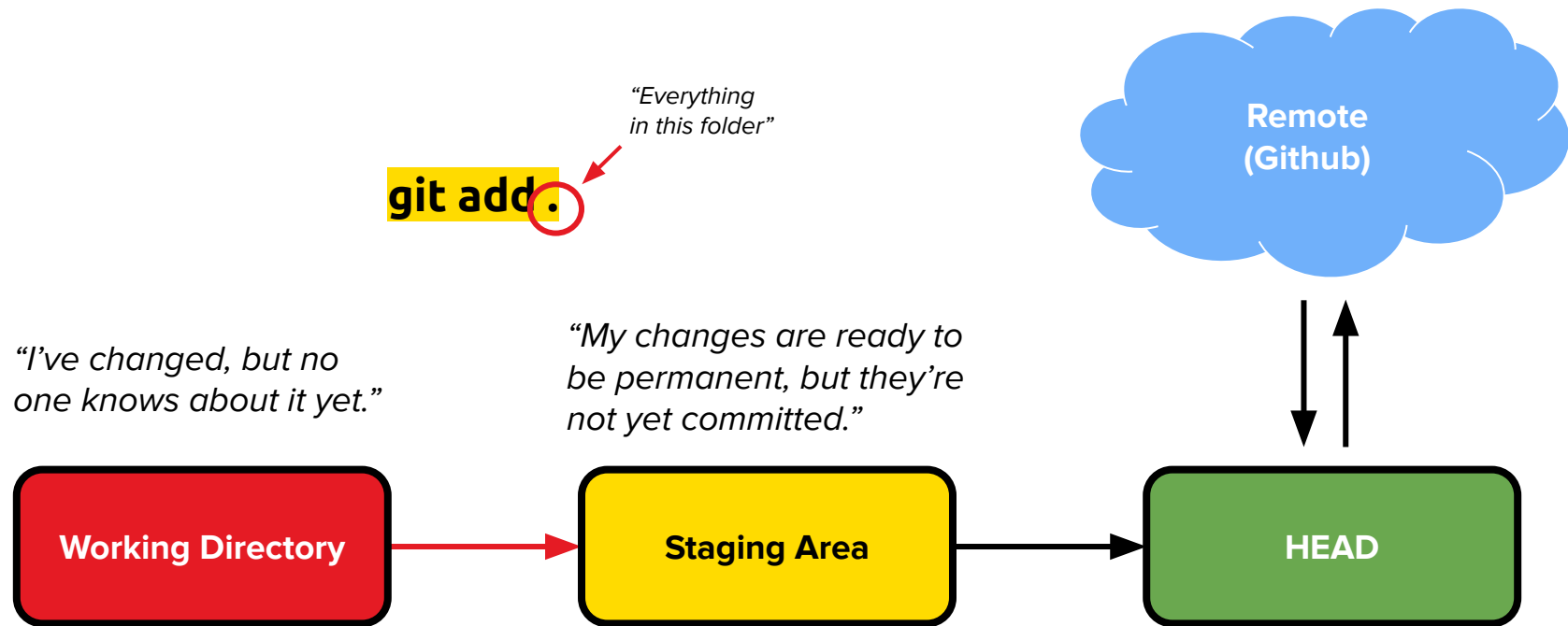
git add .

"I've changed, but no one knows about it yet."

"My changes are ready to be permanent, but they're not yet committed."



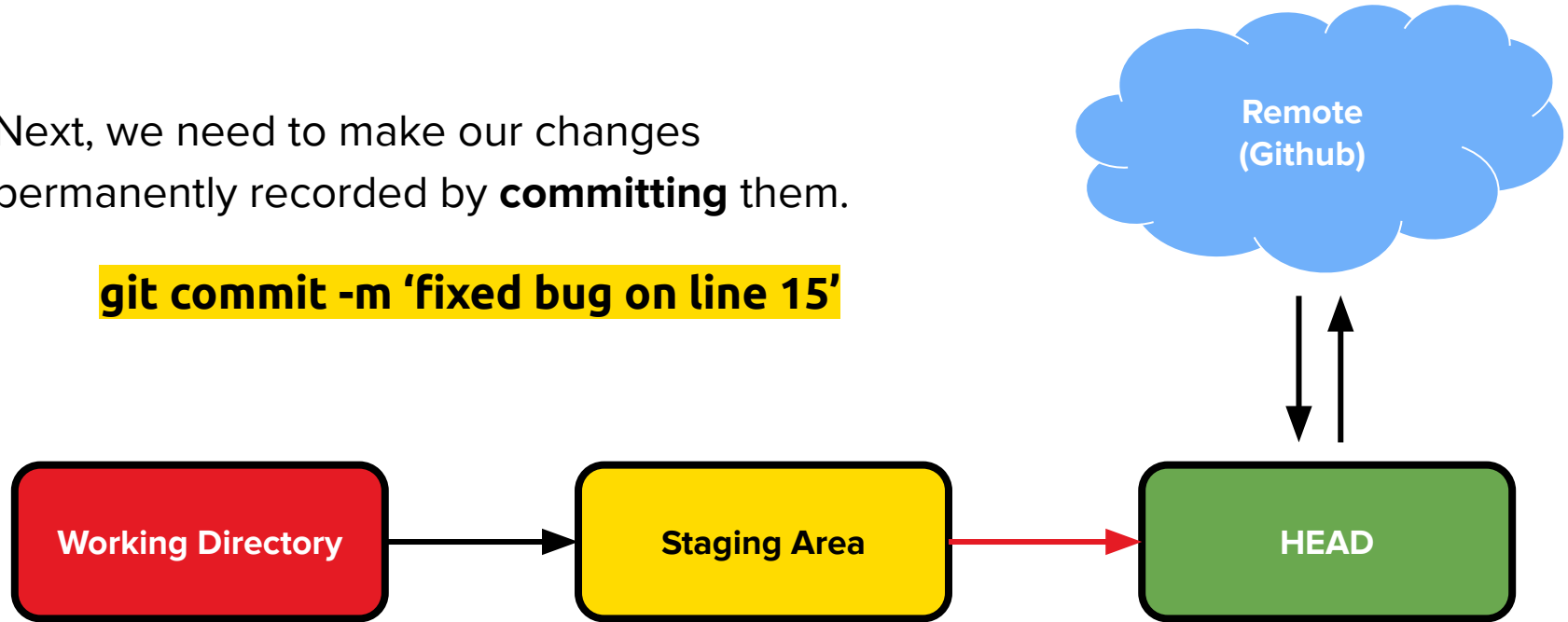
Staging Changes



Committing to Change

Next, we need to make our changes permanently recorded by **committing** them.

```
git commit -m 'fixed bug on line 15'
```



Committing to Change

```
git commit -m 'fixed bug on line 15'
```

"I've changed, but no one knows about it yet."



"My changes are ready to be permanent, but they're not yet committed."

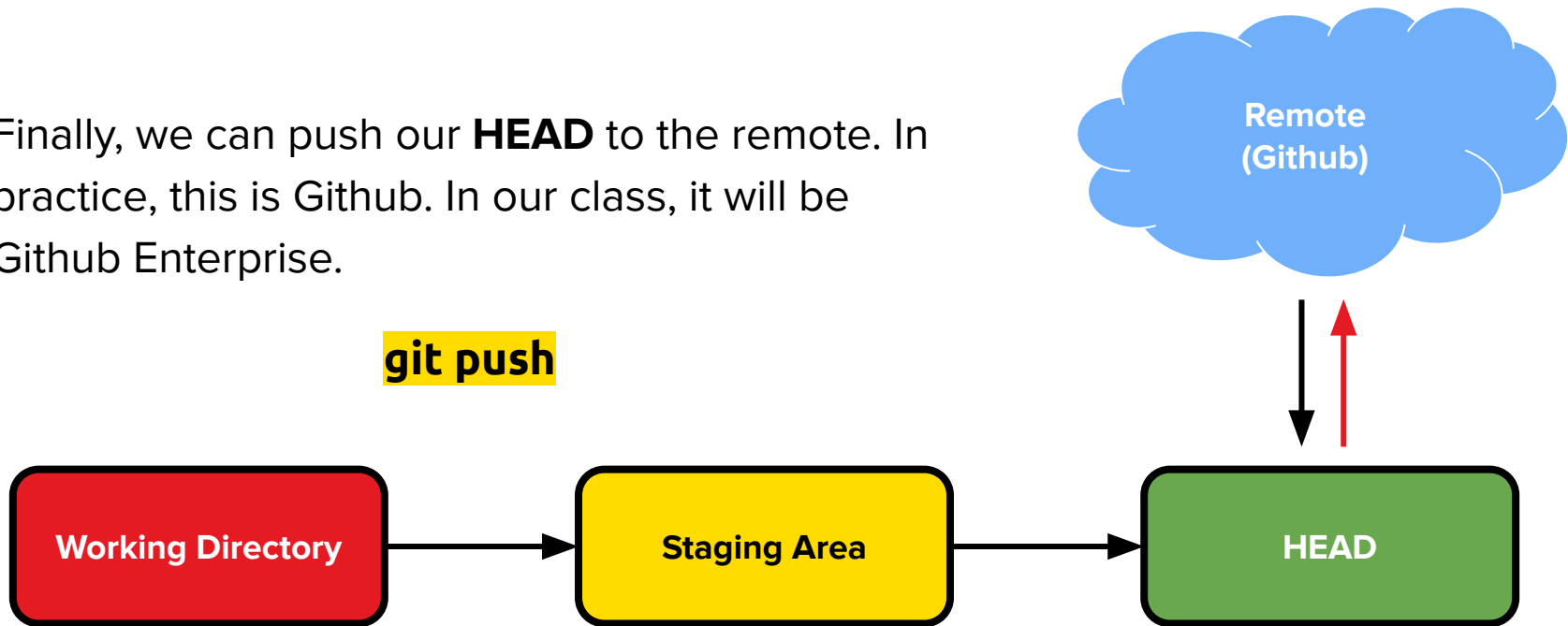


"My changes have been permanently committed, I'm ready for the world to see!"



Publishing our Work

Finally, we can push our **HEAD** to the remote. In practice, this is Github. In our class, it will be Github Enterprise.



Git Workflow

git add .	Add changes to staging area
git commit -m 'msg'	Commit changes permanently
git push	Push committed changes to Github

git status	Ask Git what's going on. You can do this anytime, and should do it often!
-------------------	---

Git Workflow

git add .	Add changes to staging area
git commit -m 'msg'	Commit changes permanently
git push	Push committed changes to Github

git status	Ask Git what's going on. You can do this anytime, and should do it often!
-------------------	---

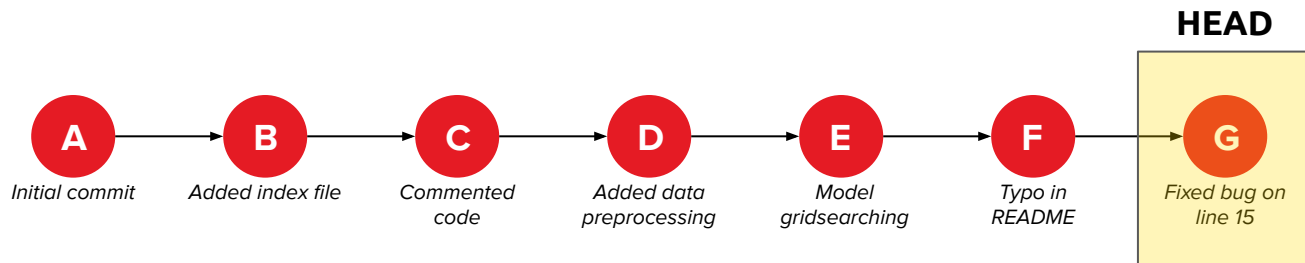


Now you:

Make another change and push it to your repo!

Git is a timeline of nodes

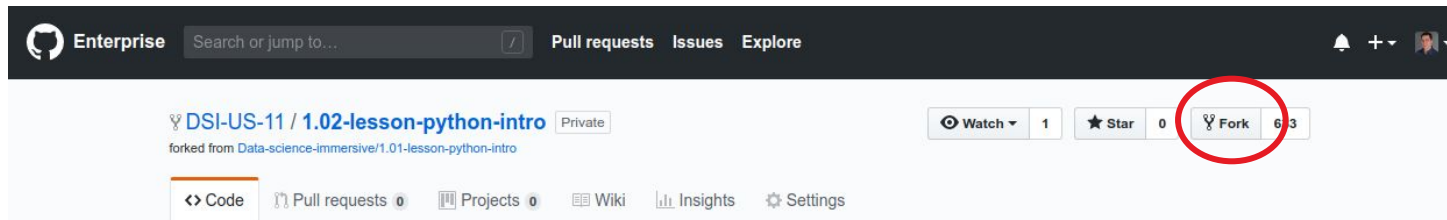
Why do we have to go through all these steps? It's so we can **time travel** if we make a mistake. In more advanced workflows, we can even **branch off** from our timeline to make different versions of our project.



— Git for DSI Lessons

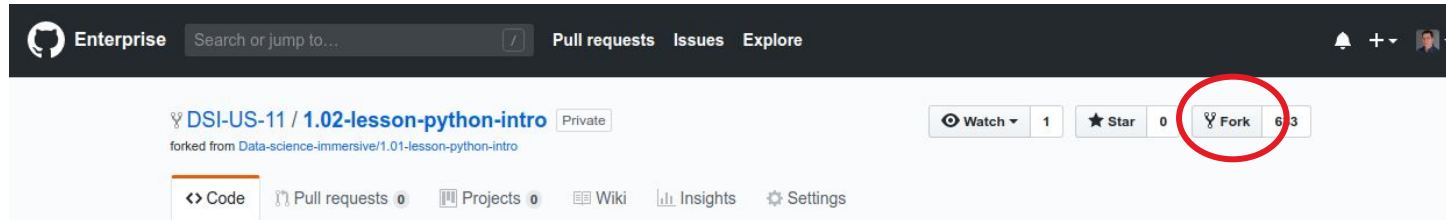
Where do lessons come from?

In our course, we also use Github as a **course management system**. Our lessons are Git repos, and you'll **fork** and **clone** them before each lesson.



Where do lessons come from?

We do this so that I (the instructor) can push the work we did in class so you can see it later, while also maintaining your own private versions backed up on Github Enterprise.



Make sure you clone *your* repo!

timbook / .02-lesson-python-intro Private

Unwatch 1 Star 0 Fork 634

Code Pull requests 0 Projects 0 Wiki Insights Settings

No description, website, or topics provided. [Manage topics](#)

21 commits 1 branch 0 releases 4 contributors

Branch: master New pull request

Create new file Upload files Find file Clone or download

This branch is even with DSI-US-11:master.

timbook updated prereqs for cc10

imgs	finished solution for cc8
solution-code	prepped for cc9
.gitignore	finished solution for cc8
README.md	updated prereqs for cc10
python-data-types-starter-code.ipynb	prepped for cc9

Clone with HTTPS ? Use SSH

Use Git or checkout with SVN using the web URL

<https://git.generalassembly.ly/timbook/>

Download ZIP

10 months ago

3 months ago

6 months ago

The Most Important Part!

Together, let's do the following:

1. **Fork** tomorrow morning's lesson
2. Create a personal GA folder and **clone** our lesson into it
3. Navigate into it and open our Jupyter Notebook
4. Run one line of code



Summary

What did we do today?

- Learned some basic command line
 - Movin' with **cd**
 - Lookin' with **ls** and **pwd**
 - Makin' with **mkdir** and **touch**
- Git
 - The Git workflow
 - How to use Git to get our course material

Cheat Sheet

<code>cd /path/to/folder</code>	Change directory (move)
<code>pwd</code>	Print working directory (where am I now?)
<code>ls</code>	List everything where I am
<code>mkdir folder-name</code>	Make a folder called folder-name
<code>touch file-name.py</code>	Make an empty text file called file-name.py

Git Workflow

git add .	Add changes to staging area
git commit -m 'msg'	Commit changes permanently
git push	Push committed changes to Github

git status	Ask Git what's going on. You can do this anytime, and should do it often!
-------------------	---